Partial credit and generalized partial credit models with latent regression

Daniel C. Furr

March 6, 2017

Table of contents

This case study uses **Stan** to fit the Partial Credit Model (PCM) and Generalized Partial Credit Model (GPCM), including a latent regression for person ability for both. Analysis is performed with **R**, making use of the **rstan** and **edstan** packages. **rstan** is the implementation of **Stan** for **R**, and **edstan** provides **Stan** models for item response theory and several convenience functions.

The **edstan** package is available on **CRAN**, but a more up to date version may often be found on Github. The following **R** code may be used to install the package from Github.

```
# Install edstan from Github rather than CRAN
install.packages("devtools")
devtools::install_github("danielcfurr/edstan")
```

The following **R** code loads the necessary packages and then sets some **rstan** options, which causes the compiled **Stan** model to be saved for future use and the MCMC chains to be executed in parallel.

```
# Load R packages
library(rstan)
library(ggplot2)
library(edstan)
library(TAM)
rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())
```

The case study uses **R** version 3.3.2, **rstan** version 2.14.1, **ggplot2** version 2.2.1, and **edstan** version 1.0.7. Also, the example data are from **TAM** version 1.995.0. Readers may wish to check the versions for their installed packages using the `packageVersion()` function.

1 Partial credit model with latent regression

1.1 Overview of the model

The PCM (Masters 1982) is appropriate for item response data that features more than two *ordered* response categories for some or all items. The items may have differing numbers of response categories. For dichotomous items (items with exactly two response categories), the partial credit model is equivalent to the Rasch model. The version presented includes a latent regression. However, the latent regression part of the model may be restricted to an intercept only, resulting in the standard partial credit model.

$$\Pr(Y_{ij} = y,\ y > 0 | \theta_j, \beta_i) = \frac{\exp \sum_{s=1}^{y} (\theta_j - \beta_{is})}{1 + \sum_{k=1}^{m_i} \exp \sum_{s=1}^{k} (\theta_j - \beta_{is})}$$

$$\Pr(Y_{ij} = y,\ y = 0 | \theta_j, \beta_i) = \frac{1}{1 + \sum_{k=1}^{m_i} \exp \sum_{s=1}^{k} (\theta_j - \beta_{is})}$$

$$\theta_j \sim \mathrm{N}(w_j' \lambda, \sigma^2)$$

Variables:

- $i = 1 \dots I$ indexes items.
- $j = 1 \dots J$ indexes persons.
- $Y_{ij} \in \{0 \dots m_i\}$ is the response of person j to item i
- m_i is simultaneously the maximum score and number of step difficulty parameters for item i.
- w_j is the vector of covariates for person j, the first element of which *must* equal one for a model intercept. w_j may be assembled into a J-by-K covariate matrix W, where K is number of elements in w_j.

Parameters:

- β_{is} is the s-th step difficulty for item i.
- θ_j is the ability for person j.
- λ is a vector of latent regression parameters of length K.
- σ^2 is the variance for the ability distribution.

Constraints:

- The last step difficulty parameter is constrained to be the negative sum of the other difficulties, resulting in the average difficulty parameter being zero.

Priors:

- $\sigma \sim \mathrm{Exp}(.1)$ is weakly informative for the person standard deviation.

- $\beta_{is} \sim \mathrm{N}(0, 9)$ is also weakly informative.
- $\lambda \sim t_3(0, 1)$, where t_3 is the Student's t distribution with three degrees of freedom, *and* the covariates have been transformed as follows: (1) continuous covariates are mean-centered and then divided by two times their standard deviations, (2) binary covariates are mean-centered and divided their maximum minus minimum values, and (3) no change is made to the constant, set to one, for the model intercept. This approach to setting priors is similar to one that has been suggested for logistic regression (Gelman et al. 2008). It is possible to adjust the coefficients back to the scales of the original covariates.

1.2 **Stan** code for a simple partial credit model

A simple **Stan** model is described before discussing the complete model, as the code for the complete model is somewhat cumbersome. The simpler model, printed below, omits the latent regression and so does not require rescaling of the person covariates or `lambda`. The mean of the person distribution is set to zero and the constraint is removed from the item difficulties, which also differs from the complete model.

```
# Print the simple PCM from the edstan package
simple_pcm_file <- system.file("extdata/pcm_simple.stan",
                               package = "edstan")
cat(readLines(simple_pcm_file), sep = "\n")
```

```
functions {
  real pcm(int y, real theta, vector beta) {
    vector[rows(beta) + 1] unsummed;
    vector[rows(beta) + 1] probs;
    unsummed = append_row(rep_vector(0.0, 1), theta - beta);
    probs = softmax(cumulative_sum(unsummed));
    return categorical_lpmf(y + 1 | probs);
  }
}
data {
  int<lower=1> I;                // # items
  int<lower=1> J;                // # persons
  int<lower=1> N;                // # responses
  int<lower=1,upper=I> ii[N];    // i for n
  int<lower=1,upper=J> jj[N];    // j for n
  int<lower=0> y[N];             // response for n; y = 0, 1 ... m_i
}
transformed data {
  int m;                         // # parameters per item (same for all items)
  m = max(y);
}
parameters {
  vector[m] beta[I];
  vector[J] theta;
  real<lower=0> sigma;
}
model {
  for(i in 1:I)
    beta[i] ~ normal(0, 3);
  theta ~ normal(0, sigma);
  sigma ~ exponential(.1);
  for (n in 1:N)
    target += pcm(y[n], theta[jj[n]], beta[ii[n]]);
}
```

The functions block includes a user-specified function `pcm()`, which accepts a response `y`, a value for `theta`, and a vector of parameters `beta` for one item. With these inputs, it returns a vector of model-predicted log probability for the response. Later, in the model block, `pcm()` is used to get the likelihood of the observed item responses.

Looking to the data block, data are fed into the model in vector form. That is, `y` is a long vector of scored item responses, and `ii` and `jj` indicate with which item and person each element in `y` is associated. These three vectors are of length `N`, which is either equal to `I` times `J` or less if there are missing responses. In the transformed data block, the variable `m` is created, which represents the number of steps per item.

In the parameters block, `beta` is declared to be `I` vectors of length `m`. Each vector in `beta` contains the step difficulties for a given item. In this simplified model, all items must have the same number of response categories. The other parameters are handled in conventional ways, with `sigma` being assigned a lower bound of zero because it is a standard deviation.

The model block indicates the priors and the likelihood. The prior for `beta` requires a loop because `beta` is an array rather than a vector. The likelihood manually increments the log posterior using the `target += ...` syntax.

1.3 **Stan** code for the partial credit model with latent regression

The PCM with latent regression will be discussed in relation to the simpler model, and both models are equivalent when the latent regression is restricted to an intercept only. The model with latent regression, which is featured in **edstan**, is printed below. It is more complicated than is typically necessary for a **Stan** model because it is written to apply sensible priors automatically for parameters associated with arbitrarily scaled covariates.

```
functions {
  real pcm(int y, real theta, vector beta) {
    vector[rows(beta) + 1] unsummed;
    vector[rows(beta) + 1] probs;
    unsummed = append_row(rep_vector(0.0, 1), theta - beta);
    probs = softmax(cumulative_sum(unsummed));
    return categorical_lpmf(y + 1 | probs);
  }
  matrix obtain_adjustments(matrix W) {
    real min_w;
    real max_w;
    int minmax_count;
    matrix[2, cols(W)] adj;
    adj[1, 1] = 0;
    adj[2, 1] = 1;
    if(cols(W) > 1) {
      for(k in 2:cols(W)) {                       // remaining columns
        min_w = min(W[1:rows(W), k]);
        max_w = max(W[1:rows(W), k]);
        minmax_count = 0;
        for(j in 1:rows(W))
          minmax_count = minmax_count + W[j,k] == min_w || W[j,k] == max_w;
        if(minmax_count == rows(W)) {       // if column takes only 2 values
          adj[1, k] = mean(W[1:rows(W), k]);
          adj[2, k] = (max_w - min_w);
        } else {                            // if column takes > 2 values
          adj[1, k] = mean(W[1:rows(W), k]);
          adj[2, k] = sd(W[1:rows(W), k]) * 2;
        }
      }
    }
    return adj;
  }
}
data {
  int<lower=1> I;               // # items
  int<lower=1> J;               // # persons
  int<lower=1> N;               // # responses
  int<lower=1,upper=I> ii[N];   // i for n
  int<lower=1,upper=J> jj[N];   // j for n
  int<lower=0> y[N];            // response for n; y = 0, 1 ... m_i
  int<lower=1> K;               // # person covariates
  matrix[J,K] W;                // person covariate matrix
}
transformed data {
  int m[I];                     // # parameters per item
  int pos[I];                   // first position in beta vector for item
  matrix[2,K] adj;              // values for centering and scaling covariates
  matrix[J,K] W_adj;            // centered and scaled covariates
  m = rep_array(0, I);
  for(n in 1:N)
    if(y[n] > m[ii[n]]) m[ii[n]] = y[n];
  pos[1] = 1;
  for(i in 2:(I))
    pos[i] = m[i-1] + pos[i-1];
```

```
  adj = obtain_adjustments(W);
  for(k in 1:K) for(j in 1:J)
      W_adj[j,k] = (W[j,k] - adj[1,k]) / adj[2,k];
}
parameters {
  vector[sum(m)-1] beta_free;
  vector[J] theta;
  real<lower=0> sigma;
  vector[K] lambda_adj;
}
transformed parameters {
  vector[sum(m)] beta;
  beta[1:(sum(m)-1)] = beta_free;
  beta[sum(m)] = -1*sum(beta_free);
}
model {
  target += normal_lpdf(beta | 0, 3);
  theta ~ normal(W_adj*lambda_adj, sigma);
  lambda_adj ~ student_t(3, 0, 1);
  sigma ~ exponential(.1);
  for (n in 1:N)
    target += pcm(y[n], theta[jj[n]],  segment(beta, pos[ii[n]], m[ii[n]]));
}
generated quantities {
  vector[K] lambda;
  lambda[2:K] = lambda_adj[2:K] ./ to_vector(adj[2,2:K]);
  lambda[1] = W_adj[1, 1:K]*lambda_adj[1:K] - W[1, 2:K]*lambda[2:K];
}
```

The complete model adds `obtain_adjustments()` to the functions block, which is used to adjust the covariate matrix. In brief, the model operates on the adjusted covariate matrix, `W_adj`, and then in the generated quantities block determines what the latent regression coefficients would be on the original scale of the covariates. For a more in depth discussion of `obtain_adjustments()` and the transformations related to the latent regression, see the Rasch and 2PL case study (http://mc-stan.org/documentation/case-studies/rasch_and_2pl.html).

In the data block, the number of covariates (plus the intercept) `K` is now required, as is the matrix of covariates `W`. Otherwise this block is the same as before. An import change in this model is that `beta` is now a single, long vector rather than an array. This set up allows items to have different numbers of steps but requires additional programming. To that end, two variables are created in the transformed data block, and these are used to access the elements in `beta` relevant to a given item: `pos` indicates the position in `beta` of the first parameter for a given item, and `m` indicates the count of parameters for an item.

The parameters `beta_free`, `theta`, `sigma`, and `lambda` are declared in the parameters block. The unconstrained item parameters are contained in `beta_free`. In the transformed parameters block, `beta` is created by appending the constrained item difficulty to `beta_free`. The model block contains the priors and the likelihood. The `target += ...` syntax for the prior on `beta` is a manual way of incrementing the log posterior used when the prior is placed on a transformed parameter.

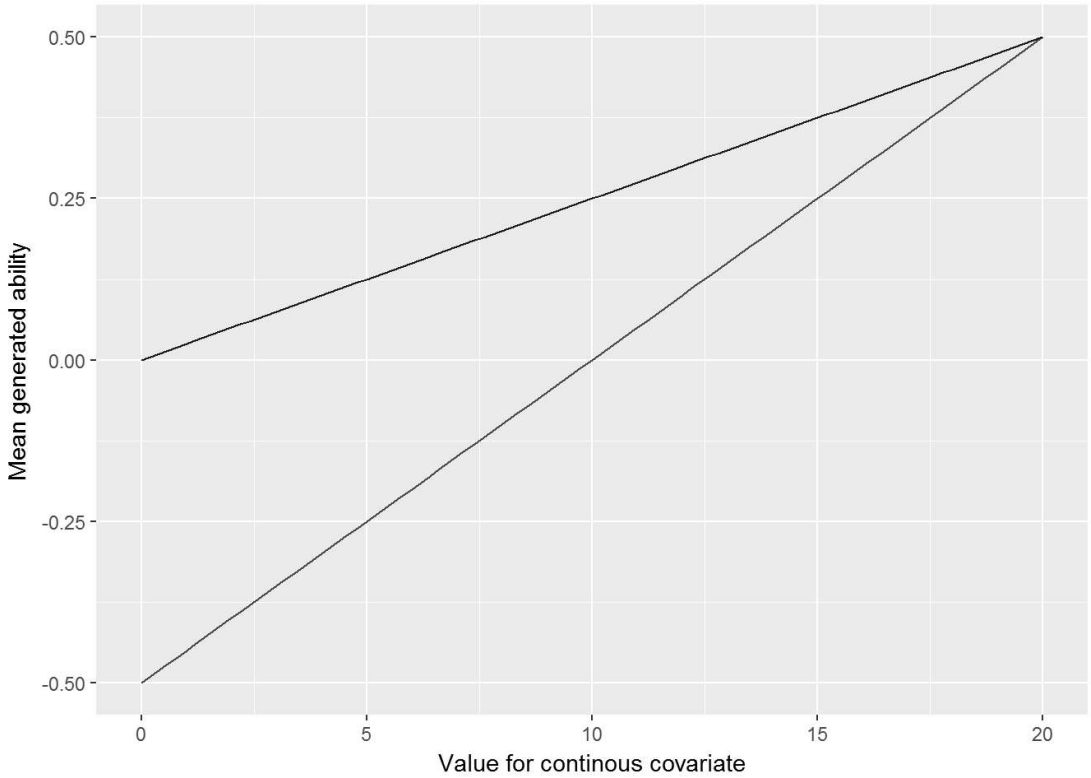
1.4 Simulation for parameter recovery

The **Stan** model is fit to a simulated dataset to evaluate it's ability to recover the generating parameter values. The **R** code that follows simulates a dataset conforming to the model.

```
# Set parameters for the simulated data
J <- 500
sigma <- 1.2
lambda <- c(-10*.05, .05, .5, -.025)
w_2 <- rnorm(J, 10, 5)
w_3 <- rbinom(J, 1, .5)
W <- cbind(1, w_2, w_3, w_2*w_3)

# Set item parameters
I <- 20
Beta_uncentered <- matrix(NA, nrow = I, ncol = 2)
Beta_uncentered[,1] <- seq(from = -1, to = 0, length.out = I)
Beta_uncentered[,2] <- Beta_uncentered[,1] + rep(c(.2, .4, .6, .8),
                                                 length.out = I)
Beta_centered <- Beta_uncentered - mean(Beta_uncentered)

# A function to simulate responses from the model
simulate_response <- function(theta, beta) {
  unsummed <- c(0, theta - beta)
  numerators <- exp(cumsum(unsummed))
  denominator <- sum(numerators)
  response_probs <- numerators/denominator
  simulated_y <- sample(1:length(response_probs) - 1, size = 1,
                        prob = response_probs)
  return(simulated_y)
}

# Calculate or sample remaining variables and parameters
N <- I*J
ii <- rep(1:I, times = J)
jj <- rep(1:J, each = I)
pcm_theta <-  rnorm(J, W %*% matrix(lambda), sigma)
pcm_y <- numeric(N)
for(n in 1:N) {
  pcm_y[n] <- simulate_response(pcm_theta[jj[n]], Beta_centered[ii[n], ])
}

# Assemble the data list using an edstan function
sim_pcm_list <- irt_data(y = pcm_y, ii = ii, jj = jj,
                         covariates = as.data.frame(W),
                         formula = NULL)
```

The simulated data consists of 20 items having 3 response categories and 500 persons. The person covariate vectors w_j include (1) a value of one for the model intercept, (2) a random draw from a normal distribution with mean of 10 and standard deviation of 5, (3) an indicator variable taking values of zero and one, and (4) an interaction between the two. These are chosen to represent a difficult case for assigning automatic priors for the latent regression coefficients. The generating coefficients λ for the latent regression are -0.5, 0.05, 0.5, and -0.025. The abilities θ are random draws from a normal distribution with a mean generated from the latent regression and a standard deviation $\sigma = 1.2$.

```
# Plot mean ability conditional on the covariates
f1 <- function(x) lambda[1] + x*lambda[2]
f2 <- function(x) lambda[1] + lambda[3] + x*(lambda[2] + lambda[4])
ggplot(data.frame(w2 = c(0, 20))) +
  aes(x = w2) +
  stat_function(fun = f1, color = "red") +
  stat_function(fun = f2, color = "blue") +
  ylab("Mean generated ability") +
  xlab("Value for continous covariate")
```



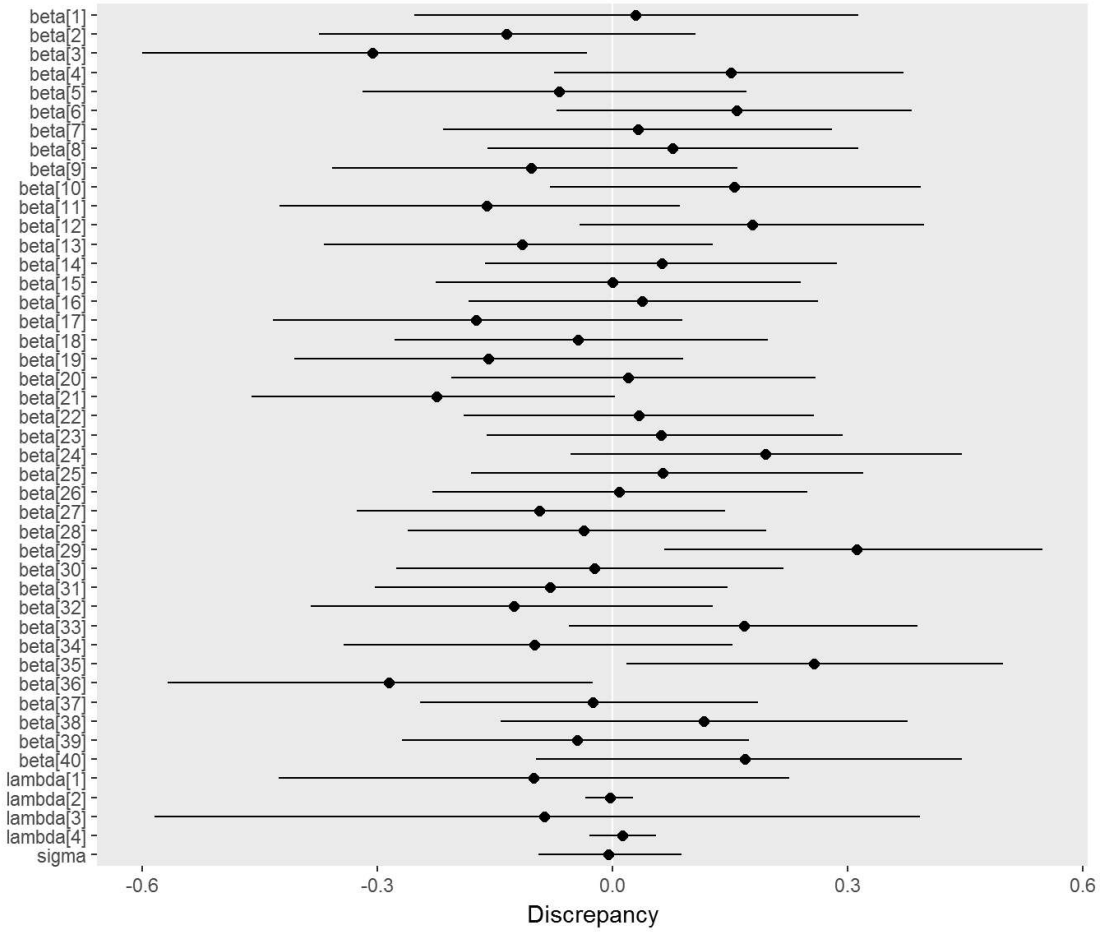
Mean of generated abilities as a function of the continuous covariate. A line is shown separately for the two groups identified by the binary variable.

The simulated dataset is next fit with **Stan** using `irt_stan()` from the **edstan** package. `irt_stan()` is merely a wrapper for `stan()` in **rstan**. Using 1,000 posterior draws per chain may be somewhat excessive as we are mainly interested in the posterior means of the parameters. However, as parameter recovery will be evaluated using the 2.5th and 97.5th percentiles of the posterior, the large number of posterior samples is warranted.

```
#Fit model to simulated data
sim_pcm_fit <- irt_stan(sim_pcm_list, model = "pcm_latent_reg.stan",
                        chains = 4, iter = 1000)
```

The highest value for $\hat{R}$ was 1.009 for all parameters and the log posterior, suggesting that the chains have converged. The **Stan** model is evaluated in terms of its ability to recover the generating values of the parameters. The R code below prepares a plot in which the points indicate the difference between the posterior means and generating values for the parameters of main interest. This difference is referred to as discrepancy. The lines indicate the 95% poster intervals for the difference, defined as the 2.5th and 97.5th percentiles of the posterior draws. Ideally, (nearly) all the 95% intervals would include zero.

```r
# Get estimated and generating values for wanted parameters
beta <- as.vector(t(Beta_centered))
pcm_generating_values <- c(beta, lambda, sigma)
pcm_estimated_values <- summary(sim_pcm_fit,
                                pars = c("beta", "lambda", "sigma"),
                                probs = c(.025, .975))
pcm_estimated_values <- pcm_estimated_values[["summary"]]

# Make a data frame of the discrepancies
pcm_discrep <- data.frame(par = rownames(pcm_estimated_values),
                          mean = pcm_estimated_values[, "mean"],
                          p025 = pcm_estimated_values[, "2.5%"],
                          p975 = pcm_estimated_values[, "97.5%"],
                          gen = pcm_generating_values)
pcm_discrep$par <- with(pcm_discrep, factor(par, rev(par)))
pcm_discrep$lower <- with(pcm_discrep, p025 - gen)
pcm_discrep$middle <- with(pcm_discrep, mean - gen)
pcm_discrep$upper <- with(pcm_discrep, p975 - gen)

# Plot the discrepancies
ggplot(pcm_discrep) +
  aes(x = par, y = middle, ymin = lower, ymax = upper) +
  scale_x_discrete() +
  labs(y = "Discrepancy", x = NULL) +
  geom_abline(intercept = 0, slope = 0, color = "white") +
  geom_linerange() +
  geom_point(size = 2) +
  theme(panel.grid = element_blank()) +
  coord_flip()
```



Discrepancies between estimated and generating parameters. Points indicate the difference between the posterior means and generating values for a parameter, and horizontal lines indicate 95% posterior intervals for the difference. Most of the discrepancies are about zero, indicating that **Stan** successfully recovers the true parameters.

2 Generalized partial credit model with latent regression

2.1 Overview of the model

The GPCM (Muraki 1992) extends the PCM by including a discrimination term. For dichotomous items (items with exactly two response categories), the generalized partial credit model is equivalent to the two-parameter logistic model. The version presented includes a latent regression. However, the latent regression may be restricted to a model intercept, resulting in the standard generalized partial credit model.

$$\Pr(Y_{ij} = y,\ y > 0 | \theta_j, \alpha_i, \beta_i) = \frac{\exp \sum_{s=1}^{y} (\alpha_i \theta_j - \beta_{is})}{1 + \sum_{k=1}^{m_i} \exp \sum_{s=1}^{k} (\alpha_i \theta_j - \beta_{is})}$$

$$\Pr(Y_{ij} = y,\ y = 0 | \theta_j, \alpha_i, \beta_i) = \frac{1}{1 + \sum_{k=1}^{m_i} \exp \sum_{s=1}^{k} (\alpha_i \theta_j - \beta_{is})}$$

$$\theta_j \sim \mathrm{N}(w_j' \lambda, 1)$$

Many aspects of the GPCM are similar to the PCM described earlier. Parameters β_i, θ_j, and λ have the same interpretation, but the GPCM adds a discrimination parameter α_i and constrains the variance of θ_j to one. The prior $\alpha_i \sim \log \mathrm{N}(1, 1)$ is added, which is weakly informative but assumes positive discriminations. The same priors are placed on β_i and λ, and the same constraint is placed on β_I.

2.2 **Stan** code for the generalized partial credit model with latent regression

The **Stan** code for the GPCM is similar to that for the PCM except for the addition of the discrimination parameters.

```
# Print the latent regression gpcm model from the edstan package
gpcm_latreg_file <- system.file("extdata/gpcm_latent_reg.stan",
                                package = "edstan")
cat(readLines(gpcm_latreg_file), sep = "\n")
```

```
functions {
  real pcm(int y, real theta, vector beta) {
    vector[rows(beta) + 1] unsummed;
    vector[rows(beta) + 1] probs;
    unsummed = append_row(rep_vector(0.0, 1), theta - beta);
    probs = softmax(cumulative_sum(unsummed));
    return categorical_lpmf(y + 1 | probs);
  }
  matrix obtain_adjustments(matrix W) {
    real min_w;
    real max_w;
    int minmax_count;
    matrix[2, cols(W)] adj;
    adj[1, 1] = 0;
    adj[2, 1] = 1;
    if(cols(W) > 1) {
      for(k in 2:cols(W)) {                       // remaining columns
        min_w = min(W[1:rows(W), k]);
        max_w = max(W[1:rows(W), k]);
        minmax_count = 0;
        for(j in 1:rows(W))
          minmax_count = minmax_count + W[j,k] == min_w || W[j,k] == max_w;
        if(minmax_count == rows(W)) {       // if column takes only 2 values
          adj[1, k] = mean(W[1:rows(W), k]);
          adj[2, k] = (max_w - min_w);
        } else {                            // if column takes > 2 values
          adj[1, k] = mean(W[1:rows(W), k]);
          adj[2, k] = sd(W[1:rows(W), k]) * 2;
        }
      }
    }
    return adj;
  }
}
data {
  int<lower=1> I;               // # items
  int<lower=1> J;               // # persons
  int<lower=1> N;               // # responses
  int<lower=1,upper=I> ii[N];   // i for n
  int<lower=1,upper=J> jj[N];   // j for n
  int<lower=0> y[N];            // response for n; y = 0, 1 ... m_i
  int<lower=1> K;               // # person covariates
  matrix[J,K] W;                // person covariate matrix
}
transformed data {
  int m[I];                     // # parameters per item
  int pos[I];                   // first position in beta vector for item
  matrix[2,K] adj;              // values for centering and scaling covariates
  matrix[J,K] W_adj;            // centered and scaled covariates
  m = rep_array(0, I);
  for(n in 1:N)
    if(y[n] > m[ii[n]]) m[ii[n]] = y[n];
  pos[1] = 1;
  for(i in 2:(I))
    pos[i] = m[i-1] + pos[i-1];
```

```
  adj = obtain_adjustments(W);
  for(k in 1:K) for(j in 1:J)
      W_adj[j,k] = (W[j,k] - adj[1,k]) / adj[2,k];
}
parameters {
  vector<lower=0>[I] alpha;
  vector[sum(m)-1] beta_free;
  vector[J] theta;
  vector[K] lambda_adj;
}
transformed parameters {
  vector[sum(m)] beta;
  beta[1:(sum(m)-1)] = beta_free;
  beta[sum(m)] = -1*sum(beta_free);
}
model {
  alpha ~ lognormal(1, 1);
  target += normal_lpdf(beta | 0, 3);
  theta ~ normal(W_adj*lambda_adj, 1);
  lambda_adj ~ student_t(3, 0, 1);
  for (n in 1:N)
    target += pcm(y[n], theta[jj[n]].*alpha[ii[n]],
                  segment(beta, pos[ii[n]], m[ii[n]]));
}
generated quantities {
  vector[K] lambda;
  lambda[2:K] = lambda_adj[2:K] ./ to_vector(adj[2,2:K]);
  lambda[1] = W_adj[1, 1:K]*lambda_adj[1:K] - W[1, 2:K]*lambda[2:K];
}
```

2.3 Simulation for parameter recovery

The **Stan** model is fit to a simulated dataset to evaluate it's ability to recover the generating parameter values. The **R** code that follows simulates a dataset conforming to the model. The step difficulties and some other elements are borrowed from the PCM simulation.

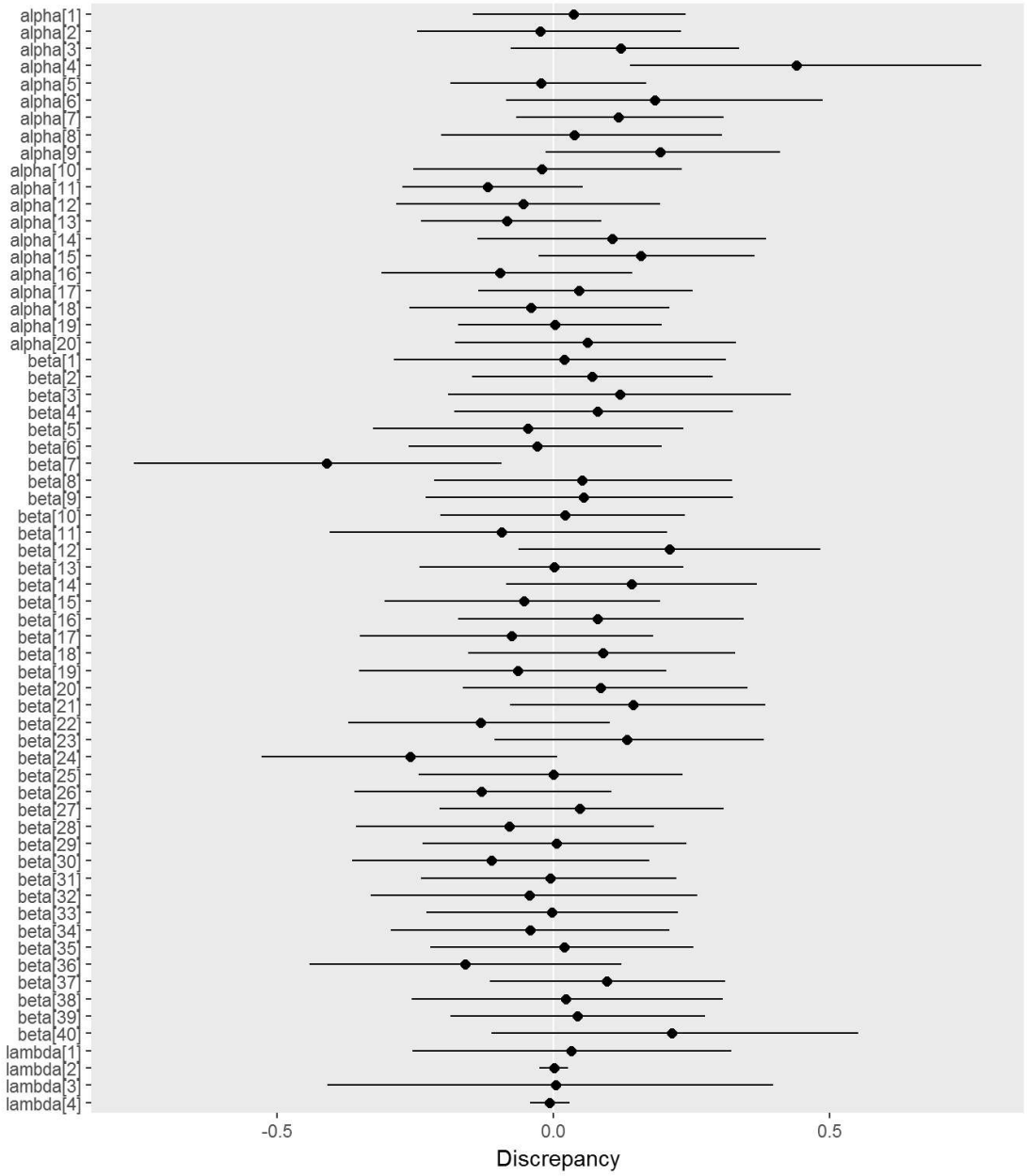
```
# Set alpha, and otherwise use parameters from the previous simulation
alpha <- rep(c(.8, 1.2),  length.out = I)

# Calculate or sample remaining variables and parameters where needed
gpcm_theta <-  W %*% matrix(lambda) + rnorm(J, 0, 1)
gpcm_y <- numeric(N)
for(n in 1:N) {
  gpcm_y[n] <- simulate_response(alpha[ii[n]]*gpcm_theta[jj[n]],
                                 Beta_centered[ii[n], ])
}

# Assemble the data list using an edstan function
sim_gpcm_list <- irt_data(y = gpcm_y, ii = ii, jj = jj,
                          covariates = as.data.frame(W),
                          formula = NULL)
```

The simulated dataset is next fit with **Stan** using `irt_stan()` from the **edstan** package.

```
# Fit model to simulated data using an edstan function
sim_gpcm_fit <- irt_stan(sim_gpcm_list, model = "gpcm_latent_reg.stan",
                         chains = 4, iter = 1000)
```

The highest value for $\hat{R}$ was 1.006 for all parameters and the log posterior. The **Stan** model is evaluated in terms of its ability to recover the generating values of the parameters. The R code below prepares a plot in which the points indicate the difference between the posterior means and generating values for the parameters of main interest. This difference is referred to as discrepancy. The lines indicate the 95% poster intervals for the difference, defined as the 2.5th and 97.5th percentiles of the posterior draws. Ideally, (nearly) all the 95% intervals would include zero.

```
# Get estimated and generating values for wanted parameters
gpcm_generating_values <- c(alpha, beta, lambda)
gpcm_estimated_values <- summary(sim_gpcm_fit,
                                 pars = c("alpha", "beta", "lambda"),
                                 probs = c(.025, .975))
gpcm_estimated_values <- gpcm_estimated_values[["summary"]]

# Make a data frame of the discrepancies
gpcm_discrep <- data.frame(par = rownames(gpcm_estimated_values),
                           mean = gpcm_estimated_values[, "mean"],
                           p025 = gpcm_estimated_values[, "2.5%"],
                           p975 = gpcm_estimated_values[, "97.5%"],
                           gen = gpcm_generating_values)
gpcm_discrep$par <- with(gpcm_discrep, factor(par, rev(par)))
gpcm_discrep$lower <- with(gpcm_discrep, p025 - gen)
gpcm_discrep$middle <- with(gpcm_discrep, mean - gen)
gpcm_discrep$upper <- with(gpcm_discrep, p975 - gen)

# Plot the discrepancies
ggplot(gpcm_discrep) +
  aes(x = par, y = middle, ymin = lower, ymax = upper) +
  scale_x_discrete() +
  labs(y = "Discrepancy", x = NULL) +
  geom_abline(intercept = 0, slope = 0, color = "white") +
  geom_linerange() +
  geom_point(size = 2) +
  theme(panel.grid = element_blank()) +
  coord_flip()
```



Discrepancies between estimated and generating parameters. Points indicate the difference between the posterior means and generating values for a parameter, and horizontal lines indicate 95% posterior intervals for the difference. Most of the discrepancies are about zero, indicating that **Stan** successfully recovers the true parameters.

3 Example application

3.1 Data

The example data are from the TIMSS 2011 mathematics assessment (Mullis et al. 2012) of Australian and Taiwanese students. For convenience, a subset of 500 students is used. The subsetted data is then divided into a person covariate matrix and an item response matrix.

```
# Attach the example dataset. The TAM package is required.
data(data.timssAusTwn.scored, package = "TAM")

# Subset the full data
select <- floor(seq(from = 1, to = nrow(data.timssAusTwn.scored),
                    length.out = 500))
subsetted_df <- data.timssAusTwn.scored[select, ]
str(subsetted_df)
```

```
## 'data.frame':    500 obs. of  14 variables:
##  $ M032166 : int  1 1 1 0 1 1 1 0 0 1 ...
##  $ M032721 : int  0 1 1 0 0 1 0 0 1 1 ...
##  $ M032757 : int  2 2 2 0 2 2 2 0 2 2 ...
##  $ M032760A: int  0 2 2 0 2 0 2 0 1 2 ...
##  $ M032760B: int  1 1 1 0 1 0 0 0 1 1 ...
##  $ M032760C: int  0 0 1 0 0 0 0 0 0 1 ...
##  $ M032761 : int  2 2 2 0 1 0 0 0 0 1 ...
##  $ M032692 : int  2 2 2 0 0 0 0 0 0 2 ...
##  $ M032626 : int  0 1 1 0 1 1 1 0 1 1 ...
##  $ M032595 : int  1 1 1 1 1 1 1 0 1 1 ...
##  $ M032673 : int  1 1 1 0 1 1 1 0 0 1 ...
##  $ IDCNTRY : int  36 36 36 36 36 36 36 36 36 36 ...
##  $ ITSEX   : int  1 1 2 2 2 2 1 1 2 1 ...
##  $ IDBOOK  : int  1 1 1 1 1 1 1 1 1 1 ...
```

The dataset is next divided into an item response matrix and a matrix of student covariates.

```
# Make a matrix of person predictors
w_mat <- cbind(intercept = rep(1, times = nrow(subsetted_df)),
               taiwan = as.numeric(subsetted_df$IDCNTRY == 158),
               female = as.numeric(subsetted_df$ITSEX == 2),
               book14 = as.numeric(subsetted_df$IDBOOK == 14))
head(w_mat)
```

```
##      intercept taiwan female book14
## [1,]         1      0      0      0
## [2,]         1      0      0      0
## [3,]         1      0      1      0
## [4,]         1      0      1      0
## [5,]         1      0      1      0
## [6,]         1      0      1      0
```

```
# Make a matrix of item responses
y_mat <- as.matrix(subsetted_df[, grep("^M", names(subsetted_df))])
head(y_mat)
```

```
##    M032166 M032721 M032757 M032760A M032760B M032760C M032761 M032692
## 1        1       0       2        0        1        0       2       2
## 4        1       1       2        2        1        0       2       2
## 8        1       1       2        2        1        1       2       2
## 11       0       0       0        0        0        0       0       0
## 15       1       0       2        2        1        0       1       0
## 18       1       1       2        0        0        0       0       0
##    M032626 M032595 M032673
## 1        0       1       1
## 4        1       1       1
## 8        1       1       1
## 11       0       1       0
## 15       1       1       1
## 18       1       1       1
```

The person covariate matrix `w_mat` has columns representing an intercept and three indicator variables for being in Taiwan (versus Australia), being female (versus male), and being assigned test booklet 14 (instead of booklet 1). The item response matrix `y_mat` contains 11 items. Neither the response matrix or person covariates contain missing data.

The following **R** code checks the maximum score per item.

```
# Maximum score for each item
apply(y_mat, 2, max)
```

```
##  M032166  M032721  M032757 M032760A M032760B M032760C  M032761  M032692
##        1        1        2        2        1        1        2        2
##  M032626  M032595  M032673
##        1        1        1
```

The above results show that the data are a mixture of dichotomous item and polytomous items with three responses categories. The first and second items are dichotomous, while the third and fourth are polytomous, for example. Consequently, the first and second items will have one step parameter each, while the third and fourth will have two each.

The data are now formatted into a data list.

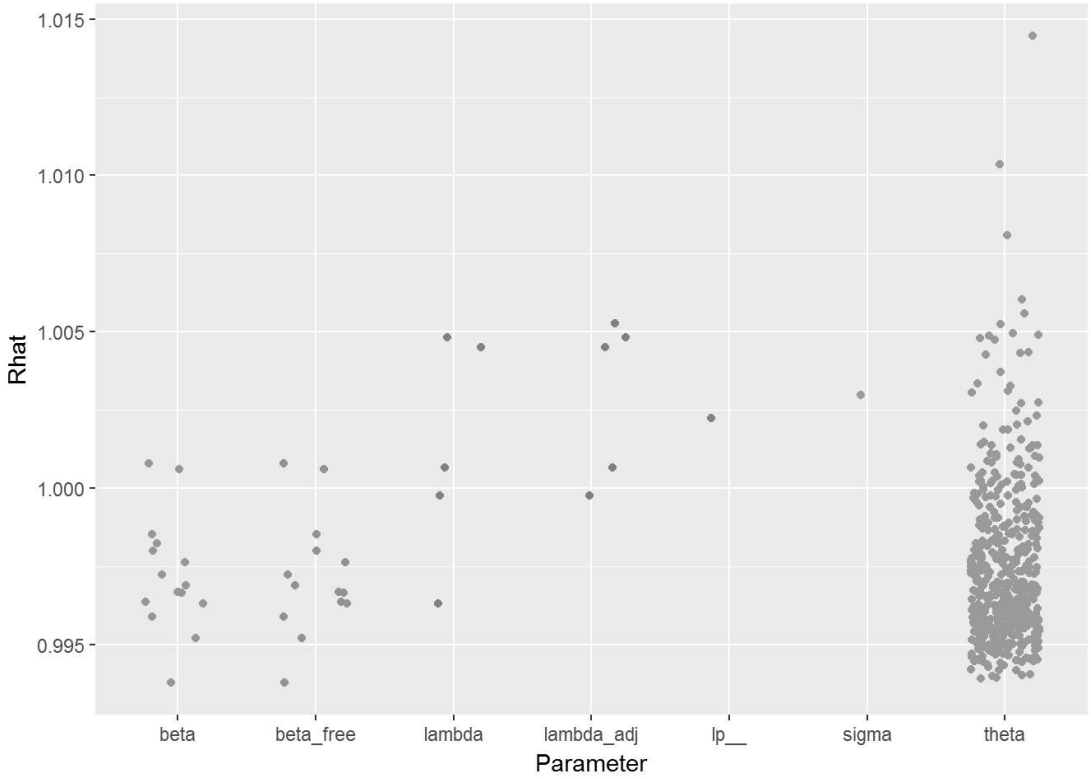
```
# Assemble data list for Stan
ex_list <- irt_data(response_matrix = y_mat, covariates = as.data.frame(w_mat),
                    formula = ~ taiwan*female + book14)
```

3.2 Partial credit model results

```
# Run Stan model
pcm_fit <- irt_stan(ex_list, model = "pcm_latent_reg.stan",
                    chains = 4, iter = 300)
```

As discussed above, convergence of the chains is assessed for every parameter, and also the log posterior density, using $\hat{R}$.

```
# Plot of convergence statistics
stan_columns_plot(pcm_fit)
```



Convergence statistics ($\hat{R}$) by parameter for the example. All values should be less than 1.1 to infer convergence.

Next we view a summary of the parameter posteriors.

```
# View table of parameter posteriors
print_irt_stan(pcm_fit, ex_list)
```

```
## Inference for Stan model: pcm_latent_reg.
## 4 chains, each with iter=300; warmup=150; thin=1;
## post-warmup draws per chain=150, total post-warmup draws=600.
##
##              mean se_mean   sd  2.5%   25%   50%   75% 97.5% n_eff Rhat
## Item 1: M032166
##   beta[1]   -1.03    0.00 0.10 -1.22 -1.10 -1.03 -0.96 -0.83   600 1.00
## Item 2: M032721
##   beta[2]    0.10    0.00 0.11 -0.10  0.02  0.11  0.18  0.31   600 1.00
## Item 3: M032757
##   beta[3]    0.68    0.01 0.23  0.20  0.51  0.68  0.83  1.14   600 1.00
##   beta[4]   -2.76    0.01 0.23 -3.24 -2.90 -2.75 -2.61 -2.32   600 1.00
## Item 4: M032760A
##   beta[5]    1.89    0.01 0.24  1.46  1.71  1.88  2.08  2.37   600 1.00
##   beta[6]   -1.82    0.01 0.24 -2.29 -1.97 -1.82 -1.64 -1.38   600 1.00
## Item 5: M032760B
##   beta[7]    0.86    0.00 0.11  0.64  0.78  0.87  0.94  1.09   600 0.99
## Item 6: M032760C
##   beta[8]    1.37    0.00 0.12  1.15  1.29  1.37  1.45  1.63   600 1.00
## Item 7: M032761
##   beta[9]    1.12    0.01 0.14  0.86  1.02  1.12  1.21  1.42   600 1.00
##   beta[10]   0.71    0.01 0.17  0.39  0.59  0.72  0.83  1.06   600 1.00
## Item 8: M032692
##   beta[11]   2.95    0.01 0.28  2.42  2.76  2.95  3.12  3.50   600 1.00
##   beta[12]  -1.51    0.01 0.29 -2.08 -1.70 -1.49 -1.31 -1.00   600 1.00
## Item 9: M032626
##   beta[13]  -0.64    0.00 0.11 -0.84 -0.71 -0.64 -0.56 -0.44   600 1.00
## Item 10: M032595
##   beta[14]  -1.05    0.00 0.12 -1.28 -1.14 -1.05 -0.98 -0.84   600 1.00
## Item 11: M032673
##   beta[15]  -0.88    0.00 0.12 -1.11 -0.96 -0.89 -0.81 -0.64   600 1.00
## Ability distribution
##   lambda[1] -0.51    0.01 0.16 -0.80 -0.62 -0.52 -0.41 -0.19   600 1.00
##   lambda[2]  2.01    0.01 0.19  1.63  1.89  2.01  2.14  2.39   600 1.00
##   lambda[3]  0.08    0.01 0.17 -0.26 -0.03  0.09  0.20  0.42   600 1.00
##   lambda[4] -0.29    0.01 0.15 -0.56 -0.38 -0.29 -0.18 -0.01   600 1.00
##   lambda[5]  0.01    0.01 0.27 -0.55 -0.17  0.00  0.18  0.52   600 1.00
##   sigma      1.40    0.00 0.07  1.28  1.36  1.40  1.45  1.54   319 1.00
##
## Samples were drawn using NUTS(diag_e) at Mon Mar 06 13:51:45 2017.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

If person covariates are unavailable, or their inclusion unwanted, the model may be fit restricting the matrix of person covariates to an intercept only. In this case, the vector `lambda` contains only one element, which will represent the mean of the ability distribution. The code below is an example of how to create the data list for this purpose.

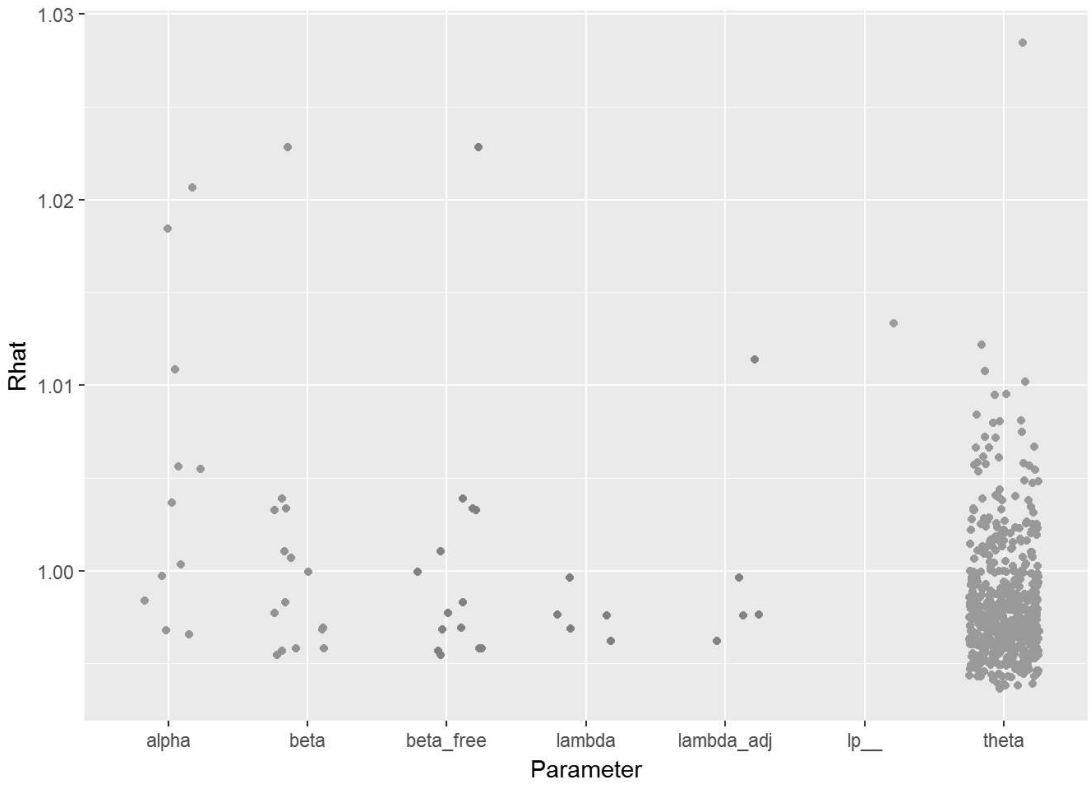
```
# Fit the example data without latent regression
noreg_list <- irt_data(response_matrix = y_mat)
noreg_fit <- irt_stan(noreg_list, model = "pcm_latent_reg.stan",
                      chains = 4, iter = 300)
```

3.3 Generalized partial credit model results

```
# Run Stan model
gpcm_fit <- irt_stan(ex_list, model = "gpcm_latent_reg.stan",
                     chains = 4, iter = 300)
```

As discussed above, convergence of the chains is assessed for every parameter, and also the log posterior density, using $\hat{R}$.

```
# Plot of convergence statistics
stan_columns_plot(gpcm_fit)
```



Convergence statistics ($\hat{R}$) by parameter for the example. All values should be less than 1.1 to infer convergence.

Next we view a summary of the parameter posteriors.

```
# View table of parameter posteriors
print_irt_stan(gpcm_fit, ex_list)
```

```
## Inference for Stan model: gpcm_latent_reg.
## 4 chains, each with iter=300; warmup=150; thin=1;
## post-warmup draws per chain=150, total post-warmup draws=600.
##
##              mean se_mean   sd  2.5%   25%   50%   75% 97.5% n_eff Rhat
## Item 1: M032166
##   alpha[1]   0.80    0.00 0.11  0.59  0.72  0.80  0.87  1.02   600 1.00
##   beta[1]   -0.94    0.00 0.11 -1.16 -1.02 -0.93 -0.86 -0.74   600 1.00
## Item 2: M032721
##   alpha[2]   0.50    0.00 0.09  0.33  0.44  0.50  0.56  0.69   600 1.00
##   beta[2]   -0.03    0.00 0.10 -0.23 -0.10 -0.03  0.03  0.16   600 1.00
## Item 3: M032757
##   alpha[3]   1.11    0.01 0.14  0.85  1.02  1.10  1.19  1.38   258 1.00
##   beta[3]    0.85    0.01 0.28  0.31  0.66  0.86  1.03  1.39   499 1.00
##   beta[4]   -2.85    0.01 0.24 -3.32 -3.00 -2.85 -2.70 -2.36   600 1.00
## Item 4: M032760A
##   alpha[4]   1.96    0.02 0.27  1.53  1.78  1.94  2.11  2.57   225 1.02
##   beta[5]    1.42    0.01 0.28  0.86  1.23  1.42  1.60  1.98   600 1.00
##   beta[6]   -1.77    0.01 0.26 -2.30 -1.95 -1.75 -1.58 -1.30   600 1.00
## Item 5: M032760B
##   alpha[5]   2.07    0.01 0.23  1.61  1.91  2.07  2.23  2.52   439 1.01
##   beta[7]    0.88    0.01 0.15  0.61  0.78  0.88  0.97  1.17   600 1.00
## Item 6: M032760C
##   alpha[6]   3.13    0.03 0.43  2.37  2.85  3.11  3.39  4.05   210 1.01
##   beta[8]    2.13    0.01 0.26  1.67  1.95  2.12  2.31  2.65   396 1.00
## Item 7: M032761
##   alpha[7]   2.69    0.03 0.36  2.07  2.43  2.67  2.94  3.44   191 1.02
##   beta[9]    0.85    0.01 0.17  0.52  0.74  0.86  0.97  1.17   600 1.00
##   beta[10]   1.75    0.02 0.34  1.12  1.52  1.75  1.98  2.43   222 1.02
## Item 8: M032692
##   alpha[8]   1.28    0.01 0.14  1.02  1.19  1.28  1.36  1.58   600 1.00
##   beta[11]   2.83    0.01 0.27  2.33  2.66  2.82  3.00  3.40   600 1.00
##   beta[12]  -1.80    0.01 0.29 -2.39 -1.98 -1.78 -1.60 -1.25   600 1.00
## Item 9: M032626
##   alpha[9]   1.70    0.01 0.18  1.36  1.56  1.69  1.82  2.06   410 1.00
##   beta[13]  -0.92    0.01 0.13 -1.17 -1.01 -0.92 -0.83 -0.67   600 1.00
## Item 10: M032595
##   alpha[10]  1.67    0.01 0.20  1.31  1.53  1.66  1.80  2.09   449 1.00
##   beta[14]  -1.37    0.01 0.15 -1.68 -1.46 -1.36 -1.26 -1.11   600 1.00
## Item 11: M032673
##   alpha[11]  1.38    0.01 0.16  1.08  1.27  1.37  1.48  1.71   600 1.01
##   beta[15]  -1.04    0.01 0.12 -1.28 -1.13 -1.04 -0.96 -0.81   484 1.00
## Ability distribution
##   lambda[1] -0.50    0.00 0.11 -0.71 -0.58 -0.50 -0.43 -0.31   600 1.00
##   lambda[2]  1.43    0.01 0.15  1.13  1.33  1.42  1.53  1.73   355 1.00
##   lambda[3]  0.06    0.01 0.13 -0.18 -0.02  0.06  0.15  0.31   600 1.00
##   lambda[4] -0.21    0.00 0.10 -0.41 -0.27 -0.21 -0.15 -0.03   600 1.00
##   lambda[5]  0.02    0.01 0.19 -0.34 -0.11  0.02  0.14  0.42   486 1.00
##
## Samples were drawn using NUTS(diag_e) at Mon Mar 06 13:52:58 2017.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

4 References

Gelman, Andrew, Aleks Jakulin, Maria Grazia Pittau, and Yu-Sung Su. 2008. “A Weakly Informative Default Prior Distribution for Logistic and Other Regression Models.” *The Annals of Applied Statistics*. JSTOR, 1360–83.

Masters, Geoff N. 1982. “A Rasch Model for Partial Credit Scoring.” *Psychometrika* 47 (2). Springer: 149–74.

Mullis, Ina V S, Michael O Martin, Pierre Foy, and Alka Arora. 2012. *TIMSS 2011 International Results in Mathematics.* ERIC.

Muraki, Eiji. 1992. “A Generalized Partial Credit Model: Application of an Em Algorithm.” *ETS Research Report Series* 1992 (1). Wiley Online Library: i–30.